Algorithm Complexity & Asymptotic Analysis

Algorithm Fundamentals & Time

What is an Algorithm?

An **algorithm** is a well-defined, step-by-step computational procedure that transforms a given set of inputs into a desired output.

Key properties include *finiteness*, *definiteness*, and *correctness* across all valid input instances.

Measuring Resource Efficiency

Algorithm performance is evaluated primarily by two metric dimensions:

- **Time Complexity:** Total operational steps executed as input size N scales.
- **Space Complexity:** Auxiliary memory allocation required during execution.

What is Asymptotic Analysis?

Asymptotic analysis evaluates an algorithm’s performance based on input size, ignoring actual running time. It measures the order of growth of time or space; for example, linear search grows linearly, while binary search grows logarithmically.

Why Asymptotic Analysis Matters

Hardware Independence: Evaluates theoretical efficiency without clock speed, OS overhead, or hardware variations.

Focus on Large Inputs: Analyzes algorithm scaling behavior as input size

$$N \to \infty$$

Dominant Term Isolation: Ignores low-order terms and constant coefficients to highlight structural growth.

Predictive Capability: Enables precise comparisons before committing code to production environments.

Asymptotic Notations

Mathematical framework for classifying growth rates: Big O, Big Omega, and Big Theta.

Formal Mathematical Definitions

Big O Notation (Upper Bound)

Represents the maximum time required (worst-case scenario).

$$f(n) = O(g(n)) \text{ if } f(n) \leq c \cdot g(n)$$

Holds for all positive constants $c > 0$ and $n \geq n_0$.

Big Theta Notation (Tight Bound)

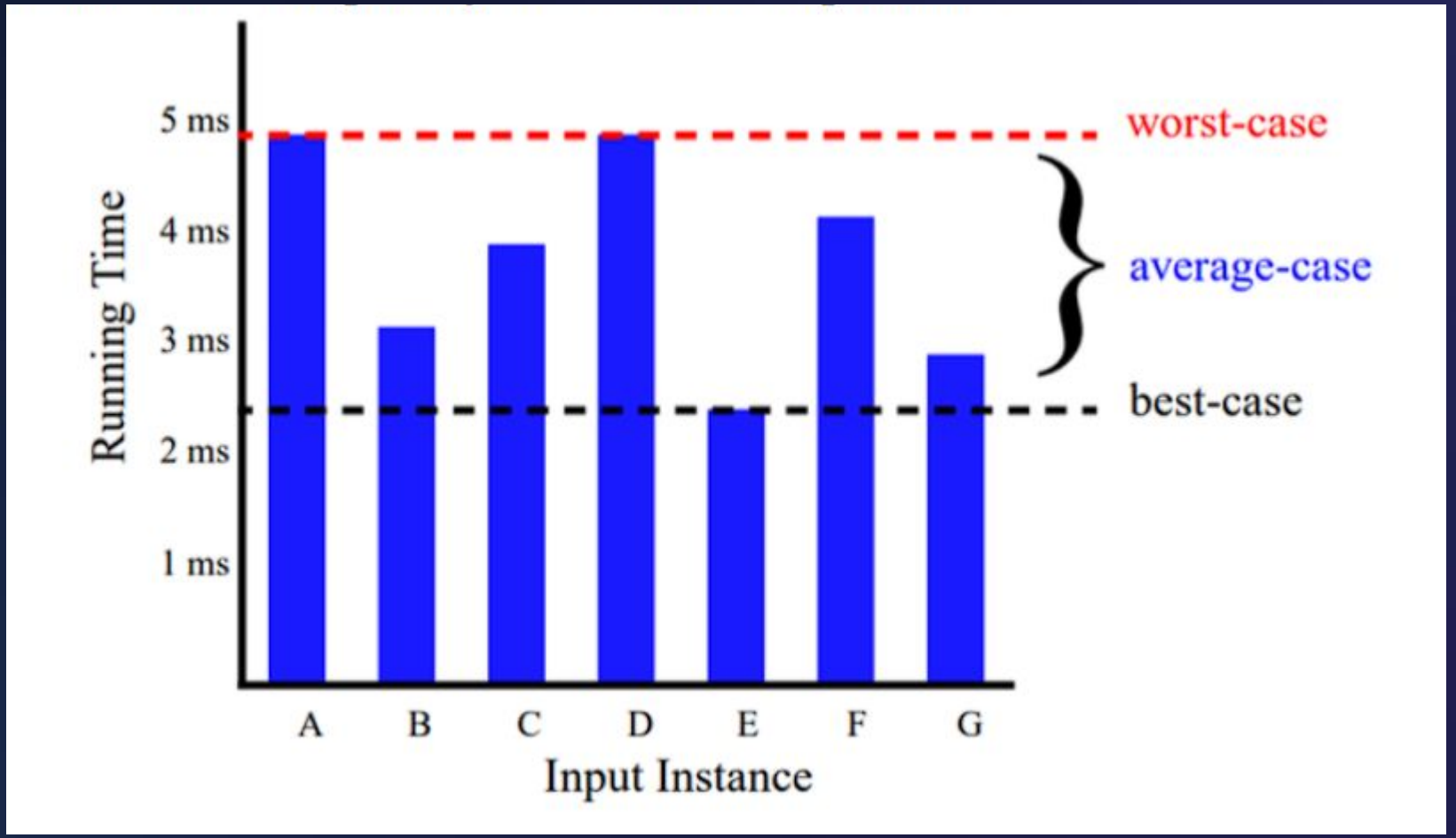
Describes exact asymptotic growth rate from above and below.

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

Requires simultaneous Big O and Big Omega conditions: $f(n) = \Theta(g(n))$.

Asymptotic Notation Comparison

Notation	Name	Bound Type	Mathematical Inequality	Practical Interpretation
$O(g(n))$	Big O	Upper Bound	$f(n) \leq c \cdot g(n)$	Worst-case scenario (at most)
$\Omega(g(n))$	Big Omega	Lower Bound	$f(n) \geq c \cdot g(n)$	Best-case scenario (at least)
$\Theta(g(n))$	Big Theta	Tight Bound	$c_1 g(n) \leq f(n) \leq c_2 g(n)$	Exact growth rate matching
$o(g(n))$	Little o	Strict Upper	$f(n) < c \cdot g(n)$	Strictly slower growth rate


Running Time
5 ms
4 ms
3 ms
2 ms
1 ms
worst-case
average-case
best-case
A
B
C
D
E
F
G
Input Instance

Operations Scale Comparison

Estimated operation counts when input size N = 1,000 demonstrating exponential scale divergence.

Common Complexity Classes

O(1) Constant

Execution time remains strictly independent of input size.

```
return array[index];
```

Examples: Hash table access, array index lookup, stack push/pop.

O(log N) Logarithmic

Execution steps cut problem space in half each iteration.

```
while (N > 1) N = N / 2;
```

Examples: Binary Search, balanced BST search operations.

O(N) Linear

Runtime scales directly in direct proportion with input length.

```
for (i = 0; i < N; i++)
```

Examples: Linear search, single loop array traversal, counting.

Advanced Complexity Classes

O(N log N) Linearithmic

Sub-divides input into subproblems and recombines results.

Standard efficient threshold for comparison-based sorting algorithms.

Examples: Merge Sort, Quick Sort (average case), Heap Sort.

O(N^2) Quadratic

Operations grow proportional to square of input length.

Commonly encountered when processing pairs or nested loops.

Examples: Bubble Sort, Insertion Sort, Selection Sort.

O(2^n) Exponential

Steps double with each increment added to input size N

Quickly becomes computationally intractable for large inputs.

Examples: Recursive Fibonacci, Set Power Subsets.

Rules for Complexity Analysis

- **Drop Constant Multipliers:** Ignore constant factors because asymptotic analysis measures long-term growth rate. For instance, $O(3N) \rightarrow O(N)$.
- **Keep Dominant Terms Only:** Lower-order terms become insignificant for large inputs. In $O\left(N^2 + 100N + 500\right)$, the quadratic term dominates: result is $O\left(N^2\right)$.
- **Add Sequential Operations:** When independent code blocks execute sequentially, add their respective complexities: $O(A + B)$.
- **Multiply Nested Loops:** When statements execute inside nested control structures, multiply outer and inner iteration bounds: $O(N \cdot M)$.

Practical Algorithm Examples

Search Algorithms Comparison

Linear Search: Inspects elements sequentially from index 0 to N-1.

Time Complexity: O(N)

Binary Search: Divide-and-conquer on sorted datasets.

Time Complexity: O(log N)

Matrix Operations Comparison

Matrix Addition: Element-wise addition across NxN grid.

Time Complexity: $O(N^2)$

Standard Matrix Multiplication: Triple nested loops.

Time Complexity: $O(N^3)$